Lesson 7 Teacher's Key: On the Move

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

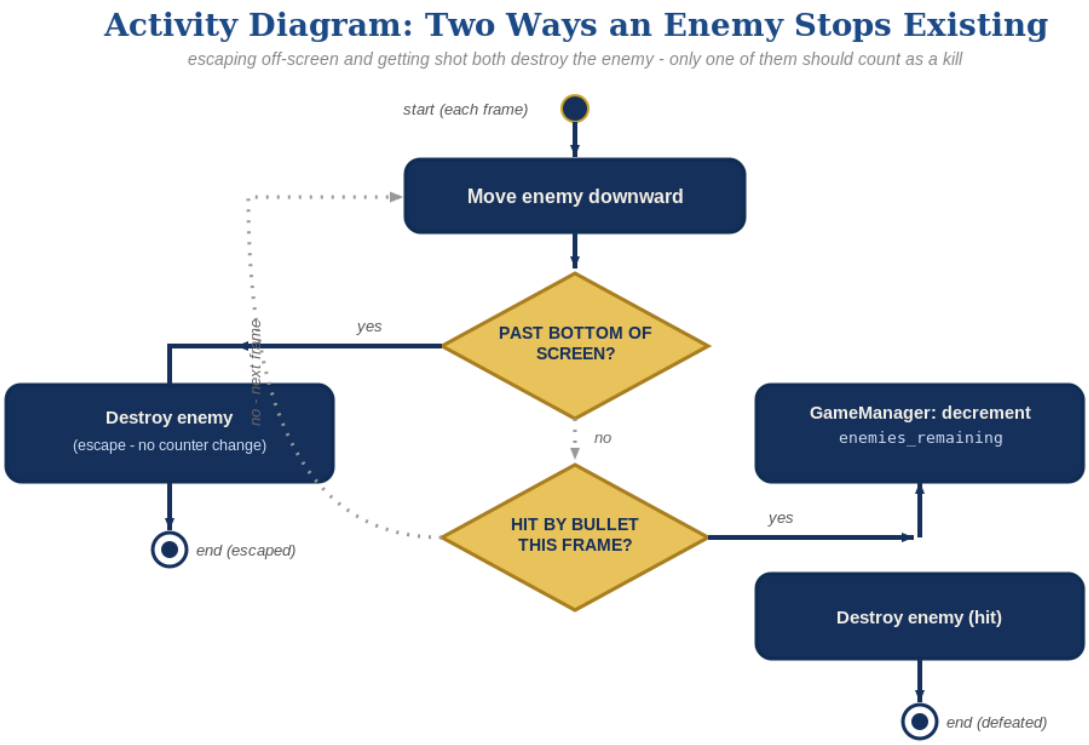
What This Lesson Teaches

A second script pathway for removing a node (queue_free() called directly from the enemy's own _process(), vs. the destroyed signal emitted only from the hitbox collision callback) - the clearest illustration in the course so far of 'not every destruction is a kill.'

The Bug Being Fixed

Godot's Lesson 6 design already emitted destroyed exclusively from _on_hitbox_area_entered(), so there is no bug to fix here - only new code to add. This engine is a good one to point to as the 'already did it right' example when explaining the GameMaker/001GC fix to the class.

Level 1 - Full Worked Solution



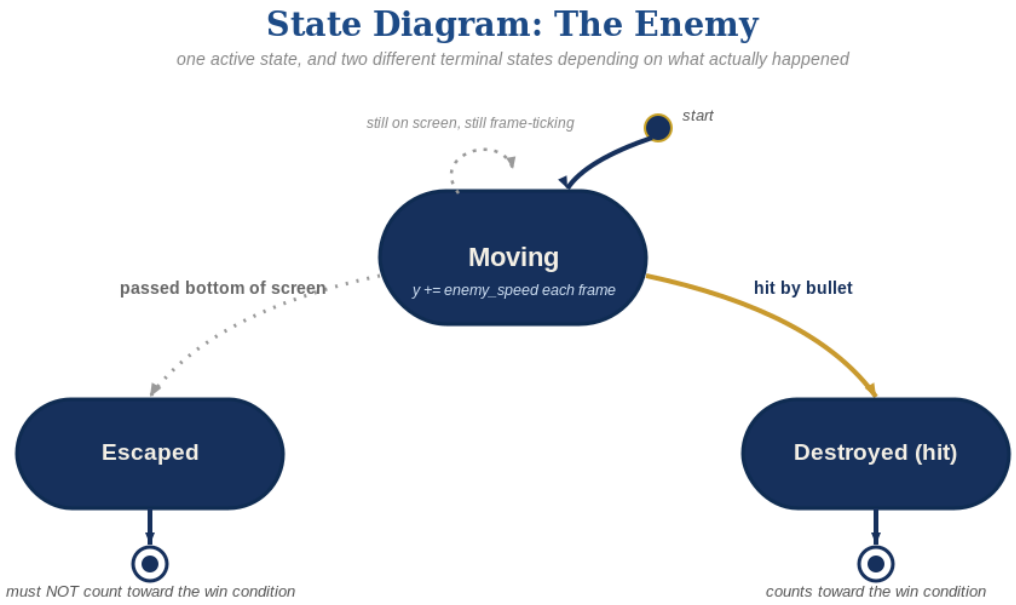
These two checks look similar - both end with "the enemy is gone" - but only the hit check is allowed to touch enemies_remaining. Attaching the counter to a generic "was destroyed" hook instead of this specific hit check is exactly the bug GameMaker and 001 Game Creator have to fix this lesson.

The general escape-vs-hit distinction behind this week's exercise - the same shape in every engine, whatever the exact code or blocks look like.

- var enemy_speed := 60.0 is a plain per-instance property; delta-scaled movement (enemy_speed * delta) keeps motion frame-rate independent, consistent with movement patterns taught since Lesson 2.
- _process(delta) runs every frame automatically once defined - no manual wiring needed, unlike Unity's Update() which behaves the same way but is worth contrasting since students are comparing engines.
- The queue_free() escape path is a completely separate code path from destroyed.emit() - _process() never touches the signal at all.

Why This Structure

- Signals are still the only thing GameManager listens to for win-counting (the main scene connects each new enemy's destroyed signal to $GameManager.on_enemy_destroyed() at spawn time, per the Lesson 6 retrofit), and _process()'s cleanup simply doesn't use that mechanism - there's no shared hook to misuse the way GameMaker's Destroy event or 001GC's Defeated trigger can be.
- This is a good moment to name the general principle explicitly for students: 'ask what specifically happened, not just that something happened, before deciding whether it counts.'



Once an enemy leaves Moving, it never comes back - both destinations are final. The whole trick this lesson teaches is making sure your code can tell these two final states apart, since only one of them is a kill.

The enemy's two possible endings this week - only one of them should count as a kill.

Level 2 / Level 3 Talking Points

- Level 3 structure talking point: this is the cleanest possible illustration of the escape-vs-hit distinction, since the two removal paths use genuinely different underlying mechanisms (direct call vs. signal) rather than the same mechanism used two different ways.
- If a student asks 'why not just check enemies_remaining inside _process() too, for safety' - explain that's redundant and actually risks re-introducing exactly the bug GameMaker/001GC has, by giving _process() any awareness of win-counting at all.

Common Student Mistakes

- Calling destroyed.emit() from _process()'s cleanup 'to be thorough' - reintroduces the exact bug this lesson is designed to avoid.
- Forgetting delta in the movement line (speed becomes frame-rate dependent, moves differently on different machines).
- Using a fixed pixel threshold that doesn't match the project's actual viewport height - remind students to check Project Settings > Display > Window if 700 doesn't look right.